

Started on	Thursday, 12 June 2025, 3:53 PM
State	Finished
Completed on	Thursday, 12 June 2025, 4:22 PM
Time taken	29 mins 37 secs
Marks	2.00/2.00
Grade	10.00 out of 10.00 (100%)

Come and Go (nguồn: UVA Online Judge, Problem 11838)

Trong một thành phố có **N** địa điểm được nối với nhau bằng **M** con đường 1 chiều và 2 chiều. Yêu cầu tối thiểu của một thành phố là **từ địa điểm này bạn phải có thể đi đến một địa điểm khác bất kỳ**.

Hãy viết chương trình kiểm tra xem các con đường của thành phố có thoả mãn yêu cầu tối thiểu này không.

Dữ liệu đầu vào có dạng như sau:

```
4 5
1 2 1
1 3 2
2 4 1
3 4 1
4 1 2
```

Trong ví dụ này, có 4 địa điểm và 5 con đường, mỗi con đường có dạng a b p, trong đó a, b là các địa điểm; và nếu p = 1, con đường đang xét là đường 1 chiều, ngược lại nó là đường 2 chiều.

Đầu vào (Input)

Dữ liệu đầu vào được nhập từ bàn phím (stdin) với định dạng:

- Dòng đầu tiên chứa 2 số nguyên N và M, tương ứng là số địa điểm và số con đường.
- M dòng tiếp theo mỗi dòng chứa 3 số nguyên a, b, p. Nếu p = 1, con đường (a, b) là con đường 1 chiều, ngược lại nếu p = 2, con đường (a, b) là con đường 2 chiều.

Đầu ra (Output)

- In ra màn hình OKIE nếu các con đường của thành phố có thoả mãn yêu cầu, ngược lại in ra NO.
- Xem thêm ví dụ bên dưới.

Gợi ý

- Xây dựng đồ thị có hướng từ dữ liệu các con đường và các địa điểm
 - Địa điểm ~ đỉnh
 - Đường 1 chiều ~ cung
 - Đường 2 chiều ~ 2 cung
- Áp dụng giải thuật kiểm tra đồ thị có liên thông mạnh hay không.

For example:

Input	Result
5 7 1 2 1 2 3 1 3 1 1 2 4 1 3 4 1 4 5 1 5 3 1	OKIE
8 10 1 2 2 1 3 1 2 8 1 3 4 1 3 5 2 4 2 1 4 7 1 4 8 1 5 7 1 6 7 2	NO

Answer: (penalty regime: 10, 20, ... %)

```
1 #include <stdio.h>
2
3 #define MAX_SIZE 100
4
5 //----- Stack-----
6 typedef int ElementType;
```

```

7
8 ▾ typedef struct {
9     ElementType data[MAX_SIZE];
10    int top_idx;
11 } Stack;
12
13 ▾ void init_stack(Stack* pS) {
14     pS->top_idx = -1;
15 }
16
17 ▾ int is_empty(Stack* pS) {
18     return pS->top_idx == -1;
19 }
20
21 ▾ void push(Stack* pS, int x) {
22     pS->top_idx++;
23     pS->data[pS->top_idx] = x;
24 }
25
26 ▾ void pop(Stack* pS) {
27     pS->top_idx--;
28 }
29
30 ▾ ElementType top(Stack* pS) {
31     return pS->data[pS->top_idx];
32 }
33
34 //-----Graph-----
35 ▾ typedef struct {
36     int n, m;
37     int A[MAX_SIZE][MAX_SIZE];
38 } Graph;
39
40 ▾ void init_graph(Graph* pG, int n) {
41     pG->n = n;
42     pG->m = 0;
43
44     for (int i = 0; i <= n; i++) {
45         for (int j = 0; j <= n; j++) {
46             pG->A[i][j] = 0;
47         }
48     }
49 }
50
51 ▾ void add_edge(Graph* pG, int u, int v) {
52     pG->A[u][v]++;
53     // if (u != v) {
54     //     pG->A[v][u]++;
55     // }
56
57     pG->m++;
58 }
59
60 ▾ int adj(Graph* pG, int u, int v) {
61     return pG->A[u][v] > 0;
62 }
63
64 //-----Strong connection-----
65 int num[MAX_SIZE];
66 int min_num[MAX_SIZE];
67 int k;
68 Stack S;
69 int on_stack[MAX_SIZE];
70 int cnt;
71
72 ▾ void SCC(Graph* pG, int u) {
73     // 1. Đánh số và đưa vào stack
74     num[u] = k;
75     min_num[u] = k;
76     k++;
77     push(&S, u);
78     on_stack[u] = 1;
79
80     // 2. Duyệt các đỉnh kề
81     for (int v = 1; v <= pG->n; v++) {
82         if (adj(pG, u, v)) {
83             // chưa duyệt
84             if (num[v] < 0) {

```

```

85         SCC(pG, v);
86         min_num[u] = min_num[v] < min_num[u] ? min_num[v] : min_num[u];
87     } else if (on_stack[v]) {
88         min_num[u] = num[v] < min_num[u] ? num[v] : min_num[u];
89     }
90 }
91 }
92
93 // 3. so sánh num và min num
94 if (num[u] == min_num[u]) {
95     cnt++;
96     int w;
97     do {
98         w = top(&S);
99         pop(&S);
100         on_stack[w] = 0;
101     } while (w != u);
102 }
103 }
104 int main() {
105     int n, m;
106     scanf("%d %d", &n, &m);
107
108     Graph G;
109     init_graph(&G, n);
110
111     for (int i = 0; i < m; i++) {
112         int u, v, t;
113         scanf("%d%d%d", &u, &v, &t);
114         add_edge(&G, u, v);
115         if (t == 2) {
116             add_edge(&G, v, u);
117         }
118     }
119
120     init_stack(&S);
121     k = 1;
122
123     // chưa duyệt
124     for (int i = 0; i <= n; i++) {
125         num[i] = -1;
126     }
127     cnt = 0;
128     for (int i = 1; i <= G.n; i++) {
129         if (num[i] < 0) {
130             SCC(&G, i);
131         }
132     }
133
134     if (cnt > 1) {
135         printf("NO");
136     } else {
137         printf("OKIE");
138     }
139 }

```

	Input	Expected	Got	
✓	5 7 1 2 1 2 3 1 3 1 1 2 4 1 3 4 1 4 5 1 5 3 1	OKIE	OKIE	✓

	Input	Expected	Got	
✓	8 10 1 2 2 1 3 1 2 8 1 3 4 1 3 5 2 4 2 1 4 7 1 4 8 1 5 7 1 6 7 2	NO	NO	✓
✓	3 2 1 2 2 1 3 2	OKIE	OKIE	✓
✓	3 2 1 2 2 1 3 1	NO	NO	✓
✓	4 2 1 2 2 3 4 2	NO	NO	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.



Trust group (nguồn: UVA Online Judge, Problem 11709)

Phòng nhân sự của tổ chức *Association of Cookie Monsters* (ACM) nhận thấy rằng gần đây hiệu quả làm việc của các nhân viên có chiều hướng giảm sút. Vì thế họ đã lấy ý kiến các nhân viên và phát hiện ra nguyên nhân của vấn đề này, đó là: sự tin cậy. Một số nhân viên không tin cậy vào các nhân viên khác trong nhóm làm việc của mình. Điều này làm giảm động lực và niềm vui trong công việc của các nhân viên.

Phòng nhân sự muốn giải quyết triệt để vấn đề này nên họ quyết định tổ chức lại các nhóm làm việc sao cho ổn định. Một nhóm làm việc sẽ ổn định khi mà những người trong nhóm tin cậy lẫn nhau. Họ đã hỏi các nhân viên và biết được những người mà một nhân viên tin cậy trực tiếp. Ngoài ra, sự tin cậy có tính bắt cầu: *nếu A tin cậy B và B tin cậy C thì A cũng sẽ tin cậy C*. Dĩ nhiên, một nhân viên sẽ tự tin cậy chính bản thân mình. Tuy nhiên, cần chú ý là sự tin cậy lại không có tính đối xứng: *A tin cậy B thì không nhất thiết B phải tin cậy A*.

Phòng nhân sự muốn tổ chức thành ít nhóm nhất có thể. Hãy lập trình để giúp họ.

Giả sử các nhân viên được đánh số là 1, 2, ..., n.

Đầu vào (Input)

Dữ liệu đầu vào được nhập từ bàn phím theo định dạng:

- Dòng đầu tiên chứa 2 số nguyên n và m, tương ứng là số nhân viên và số cặp tin cậy.
- m dòng tiếp theo, mỗi dòng chứa 2 số nguyên a b, nói rằng người a tin cậy vào người b.

Đầu ra (Output)

- In ra màn hình số lượng nhóm ít nhất mà những người trong nhóm đều tin cậy lẫn nhau
- Xem thêm trong phần ví dụ

For example:

Input	Result
5 7 1 2 2 3 3 1 2 4 3 4 4 5 5 3	1
8 13 1 2 1 3 2 1 2 8 3 4 3 5 4 2 4 7 4 8 5 3 5 7 6 7 7 6	3

Answer: (penalty regime: 10, 20, ... %)

```
1 #include <stdio.h>
2
3 #define MAX_SIZE 100
4
5 //----- Stack-----
6 typedef int ElementType;
7
8 typedef struct {
9     ElementType data[MAX_SIZE];
10     int top_idx;
11 } Stack;
12
13 void init_stack(Stack* pS) {
14     pS->top_idx = -1;
15 }
16
```

```

17 int is_empty(Stack* pS) {
18     return pS->top_idx == -1;
19 }
20
21 void push(Stack* pS, int x) {
22     pS->top_idx++;
23     pS->data[pS->top_idx] = x;
24 }
25
26 void pop(Stack* pS) {
27     pS->top_idx--;
28 }
29
30 ElementType top(Stack* pS) {
31     return pS->data[pS->top_idx];
32 }
33
34 //-----Graph-----
35 typedef struct {
36     int n, m;
37     int A[MAX_SIZE][MAX_SIZE];
38 } Graph;
39
40 void init_graph(Graph* pG, int n) {
41     pG->n = n;
42     pG->m = 0;
43
44     for (int i = 0; i <= n; i++) {
45         for (int j = 0; j <= n; j++) {
46             pG->A[i][j] = 0;
47         }
48     }
49 }
50
51 void add_edge(Graph* pG, int u, int v) {
52     pG->A[u][v]++;
53     // if (u != v) {
54     //     pG->A[v][u]++;
55     // }
56
57     pG->m++;
58 }
59
60 int adj(Graph* pG, int u, int v) {
61     return pG->A[u][v] > 0;
62 }
63
64 //-----Strong connection-----
65 int num[MAX_SIZE];
66 int min_num[MAX_SIZE];
67 int k;
68 Stack S;
69 int on_stack[MAX_SIZE];
70 int cnt;
71
72 void SCC(Graph* pG, int u) {
73     // 1. Đánh số và đưa vào stack
74     num[u] = k;
75     min_num[u] = k;
76     k++;
77     push(&S, u);
78     on_stack[u] = 1;
79
80     // 2. Duyệt các đỉnh kề
81     for (int v = 1; v <= pG->n; v++) {
82         if (adj(pG, u, v)) {
83             // chưa duyệt
84             if (num[v] < 0) {
85                 SCC(pG, v);
86                 min_num[u] = min_num[v] < min_num[u] ? min_num[v] : min_num[u];
87             } else if (on_stack[v]) {
88                 min_num[u] = num[v] < min_num[u] ? num[v] : min_num[u];
89             }
90         }
91     }
92
93     // 3. so sánh num và min num
94     if (num[u] == min_num[u]) {

```

```

95     cnt++;
96     int w;
97     do {
98         w = top(&S);
99         pop(&S);
100         on_stack[w] = 0;
101     } while (w != u);
102 }
103 }
104 int main() {
105     int n, m;
106     scanf("%d %d", &n, &m);
107
108     Graph G;
109     init_graph(&G, n);
110
111     for (int i = 0; i < m; i++) {
112         int u, v;
113         scanf("%d%d", &u, &v);
114         add_edge(&G, u, v);
115     }
116
117     init_stack(&S);
118     k = 1;
119
120     // chưa duyệt
121     for (int i = 0; i <= n; i++) {
122         num[i] = -1;
123     }
124     cnt = 0;
125     for (int i = 1; i <= G.n; i++) {
126         if (num[i] < 0) {
127             SCC(&G, i);
128         }
129     }
130 }
131
132 printf("%d", cnt);
133 }

```

	Input	Expected	Got	
✓	5 7 1 2 2 3 3 1 2 4 3 4 4 5 5 3	1	1	✓
✓	8 13 1 2 1 3 2 1 2 8 3 4 3 5 4 2 4 7 4 8 5 3 5 7 6 7 7 6	3	3	✓

Passed all tests! ✓

Question author's solution (C):

```
1 #include <stdio.h>
```



```

1  #include <stdio.h>
2
3  /* Khai báo CTDL Graph*/
4  #define MAX_N 100
5  typedef struct {
6      int n, m;
7      int A[MAX_N][MAX_N];
8  } Graph;
9
10 void init_graph(Graph *pG, int n) {
11     pG->n = n;
12     pG->m = 0;
13     for (int u = 1 ; u <= n; u++)
14         for (int v = 1 ; v <= n; v++)
15             pG->A[u][v] = 0;
16 }
17
18 void add_edge(Graph *pG, int u, int v) {
19     pG->A[u][v] += 1;
20     //if (u != v)
21     //    pG->A[v][u] += 1;
22
23     if (pG->A[u][v] > 1)
24         printf("da cung (%d, %d)\n", u, v);
25     if (u == v)
26         printf("khuyen %d\n", u);
27
28
29     pG->m++;
30 }
31
32 int adjacent(Graph *pG, int u, int v) {
33     return pG->A[u][v] > 0;
34 }
35
36
37 #define MAX_SIZE 100
38 typedef int ElementType;
39 typedef struct {
40     ElementType data[MAX_SIZE];
41     int top_idx;
42 } Stack;
43 /* Hàm khởi tạo ngăn xếp rỗng */
44 void make_null_stack(Stack *pS) {
45     pS->top_idx = -1;
46 }
47 /* Hàm thêm phần tử u vào đỉnh ngăn xếp */
48 void push(Stack *pS, ElementType u) {
49     pS->top_idx++;
50     pS->data[pS->top_idx] = u;
51 }
52 /* Hàm xem phần tử trên đỉnh ngăn xếp */
53 ElementType top(Stack *pS) {
54     return pS->data[pS->top_idx];
55 }
56 /* Hàm xóa bỏ phần tử trên đỉnh ngăn xếp */
57 void pop(Stack *pS) {
58     pS->top_idx--;
59 }
60 /* Hàm kiểm tra ngăn xếp rỗng */
61 int empty(Stack *pS) {
62     return pS->top_idx == -1;
63 }
64
65
66
67 int min(int a, int b) {
68     return a < b ? a : b;
69 }
70
71
72 int num[MAX_N], min_num[MAX_N];
73 int k;
74 Stack S;
75 int on_stack[MAX_N];
76 int nb_cnt;
77
78
79 /* Đọc dữ liệu từ tệp và xử lý

```

```

79 //Duyệt đồ thị bắt đầu từ đỉnh u
80 void SCC(Graph *pG, int u) {
81     //1. Đánh số u, đưa u vào ngăn xếp S
82     num[u] = min_num[u] = k; k++;
83     push(&S, u);
84     on_stack[u] = 1;
85     //2. Xét các đỉnh kề của u
86     for (int v = 1; v <= pG->n; v++) {
87         if (adjacent(pG, u, v)) {
88             if (num[v] < 0) {
89                 SCC(pG, v);
90                 min_num[u] = min(min_num[u], min_num[v]);
91             } else if (on_stack[v])
92                 min_num[u] = min(min_num[u], num[v]);
93         }
94     }
95     //3. Kiểm tra u có phải là đỉnh khớp
96     if (num[u] == min_num[u]) {
97         //printf("Tim duoc BPLT manh, %d la dinh khop.\n", u);
98         nb_cnt++;
99         int w;
100         do { //Lấy các đỉnh trong S ra cho đến khi gặp u
101             w = top(&S);
102             pop(&S);
103             on_stack[w] = 0;
104             //printf("Lay %d.\n", w);
105         } while (w != u);
106     }
107 }
108 }
109
110
111
112
113 int main() {
114     //1. Khai báo đồ thị G
115     Graph G;
116     //2. Đọc dữ liệu và dựng đồ thị
117     int n, m, u, v;
118     scanf("%d%d", &n, &m);
119     init_graph(&G, n);
120     for (int e = 0; e < m; e++) {
121         scanf("%d%d", &u, &v);
122         add_edge(&G, u, v);
123     }
124
125     for (int u = 1; u <= G.n; u++)
126         num[u] = -1;
127
128     //3. Duyệt toàn bộ đồ thị để kiểm tra chu trình
129     k = 1; //1b. Tất cả đều chưa duyệt
130     make_null_stack(&S); //1c. Làm rỗng ngăn xếp
131     //2. Duyệt toàn bộ đồ thị để tìm BPLT mạnh
132     nb_cnt = 0;
133     for (int u = 1; u <= G.n; u++)
134         if (num[u] == -1) //u chưa duyệt
135             SCC(&G, u); //duyet nó
136
137
138     printf("%d\n", nb_cnt);
139
140     return 0;
141 }
142
143
144

```

Correct

Marks for this submission: 1.00/1.00.



